CS24: Binary Heap Lecture Notes

Motivation

```
Real-world problem  → Algorithm   →   ADT          → Data structure → Representation
Navigation            Dijkstra's      Priority Queue   Binary Heap      Vector
Data Compression      Huffman Coding  Priority Queue   Binary Heap      Vector
```

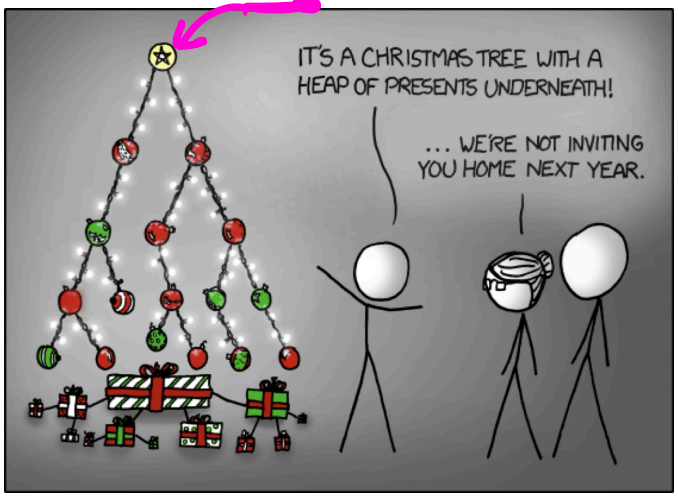
Last lecture we used `priority_queue` as a black box. Its run times:

Operation	Run time
`top()`	O(1) // highest priority (min or max)
`push(x)`	O(log n)
`pop()`	O(log n)

And unlike a BST (non-self balancing kind like the one you implemented) — where inserting sorted data degenerates the tree into a linked list and collapses search to O(n) — a binary heap **always** guarantees these run times. No degenerate case.

PQ → Binary Heap

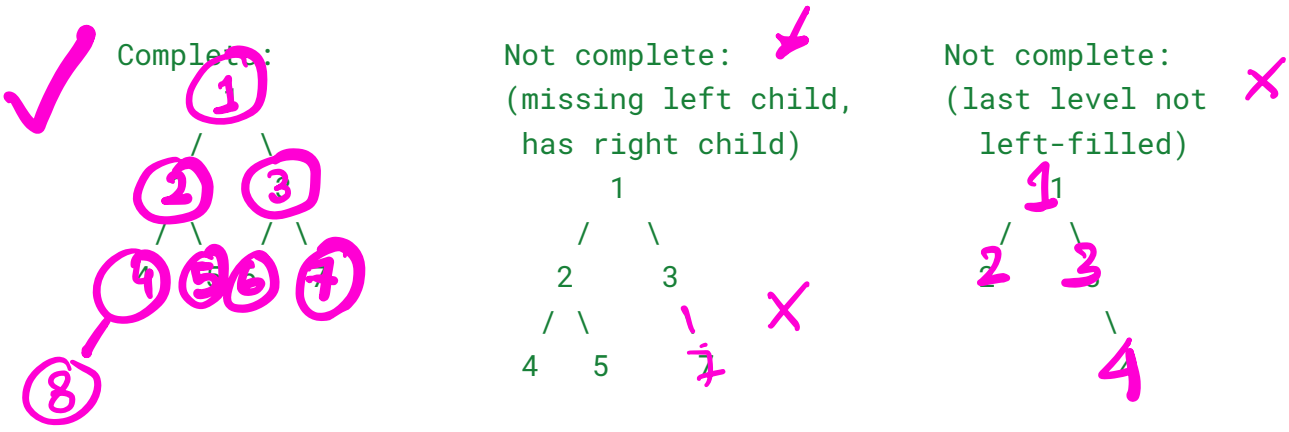
Question: Why is `top()` O(1) but `push()` and `pop()` are O(log n)?



1. Complete Binary Trees

A **binary tree** is *complete* if:

- Every level is fully filled, **except possibly the last**
- The last level is filled from **left to right** — no gaps



Key fact: A complete binary tree with n nodes has height $\lfloor \log_2 n \rfloor$.

2. Binary Heap: Two Properties

A **binary heap** is a complete binary tree where every node also satisfies the **heap property**.

(1) Shape property: The tree is a complete binary tree.

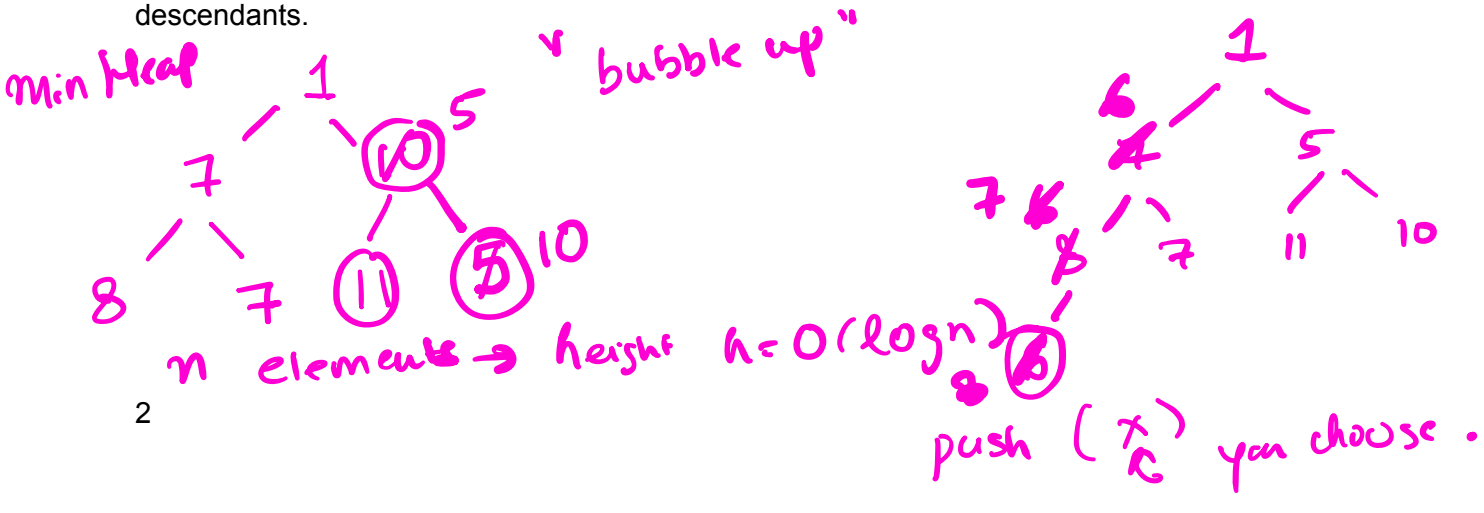
(2) Heap property:

- **Min-heap:** for every node x, `key(x) ≤ key(children of x)`
- **Max-heap:** for every node x, `key(x) ≥ key(children of x)`

Fill in: The heap property guarantees the root always holds the _____.

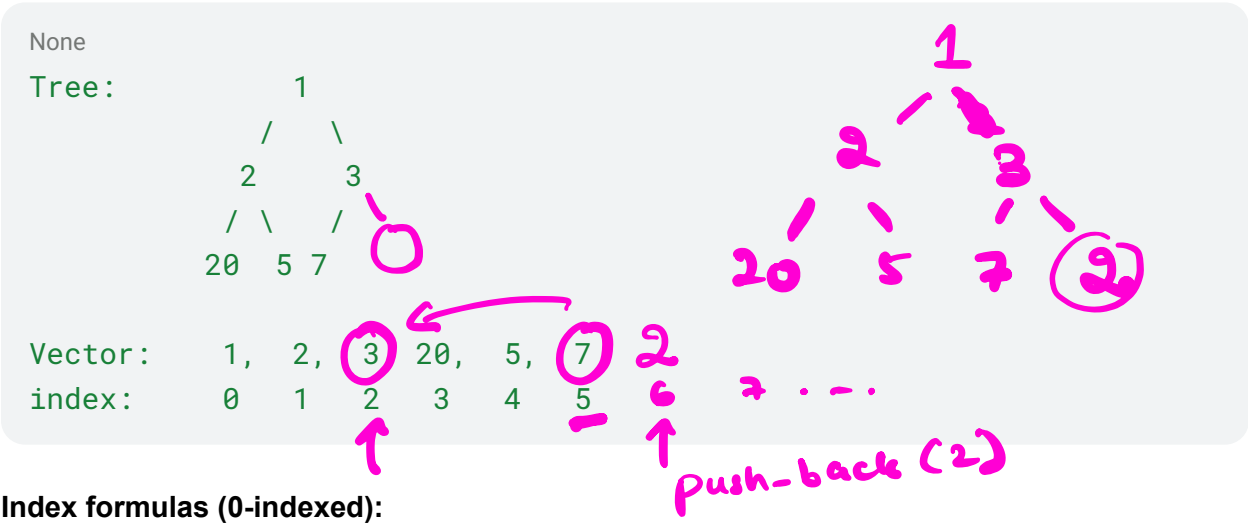
This is why `top()` is O(1) — the answer is always sitting right there.

Note: The heap property is **local** — each node only cares about its children, not all descendants.



3. Complete binary tree —Vector Representation

A complete binary tree can be stored as a **plain vector** using level-order (BFS) order. No struct to represent nodes, no pointers. Just a vector!



Index formulas (0-indexed):

	Formula
Root	index `0`
Parent of node at index `i`	`⌊(i - 1) / 2⌋`
Left child of node at index `i`	`2i + 1`
Right child of node at index `i`	`2i + 2`

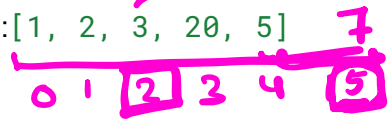
Quick checks — answer these on the vector above:

Question	**Answer**
What is the root?	key = _____ at index _____
Where is the next open spot for a new insertion?	index _____
Parent of node at index 4 (key 5)?	index _____ , key _____
Children of node at index 1 (key 2)?	left: index _____ right: index _____

`push(x)` — O(log n)

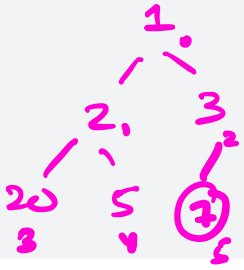
```
None
procedure push(x):
    1. Insert x in the first open spot in the tree
    2. while x has a parent AND parent(x) < x:This loop is the "bubble up"
           swap(x, parent(x))
```

Activity 1: Trace the operation `push(7)` on the given binary heap:`[1, 2, 3, 20, 5]`
Show how the vector evolves during the "bubble up" operation



Write the sequence of indices in the vector that were involved in a swap during the "bubble up"

```
None
A. 5, 4, 3, 2, 1, 0
B. 5, 4, 2, 1, 0
C. 5, 3, 1
D. 5, 2, 0
E. None of these
```



Why is the running time of push O(log n)?

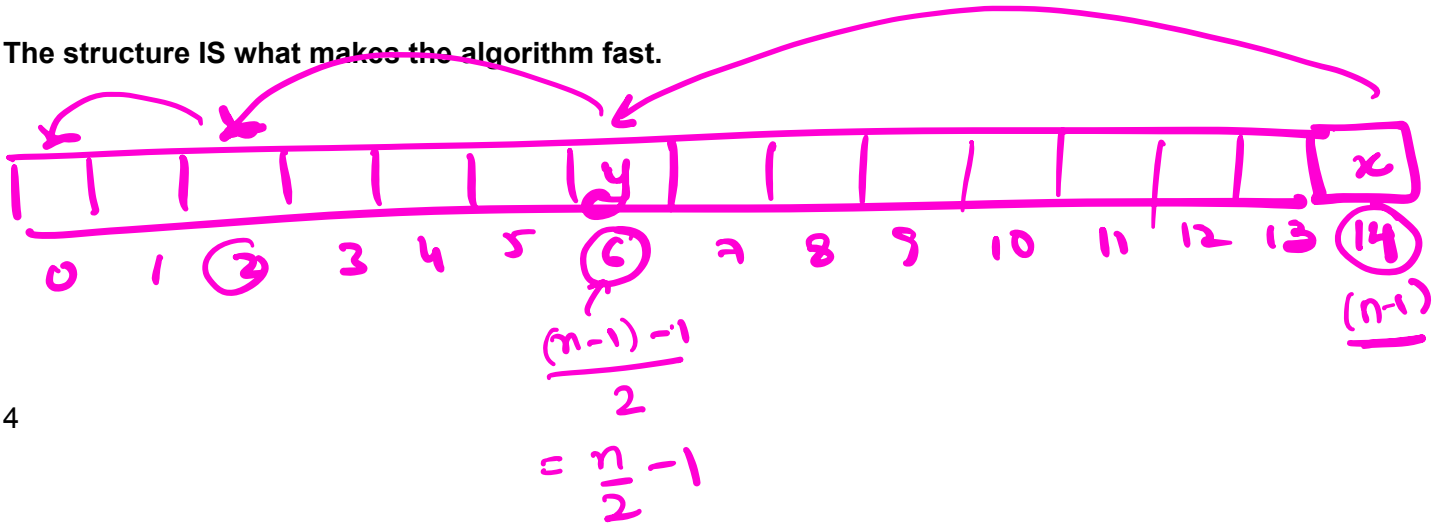
The next open spot is always the **next index in the vector** — just `v.push_back(x)`.

Traversing up/down is just arithmetic, no pointer chasing:

```
None
Bubble up:   i  →  ⌊(i-1)/2⌋        repeat until heap property holds
Bubble down: i  →  2i+1 or 2i+2     pick the better child, repeat
```

Each step is O(1). At most $\lfloor \log_2 n \rfloor$ steps. **That's where O(log n) comes from.**

The structure IS what makes the algorithm fast.



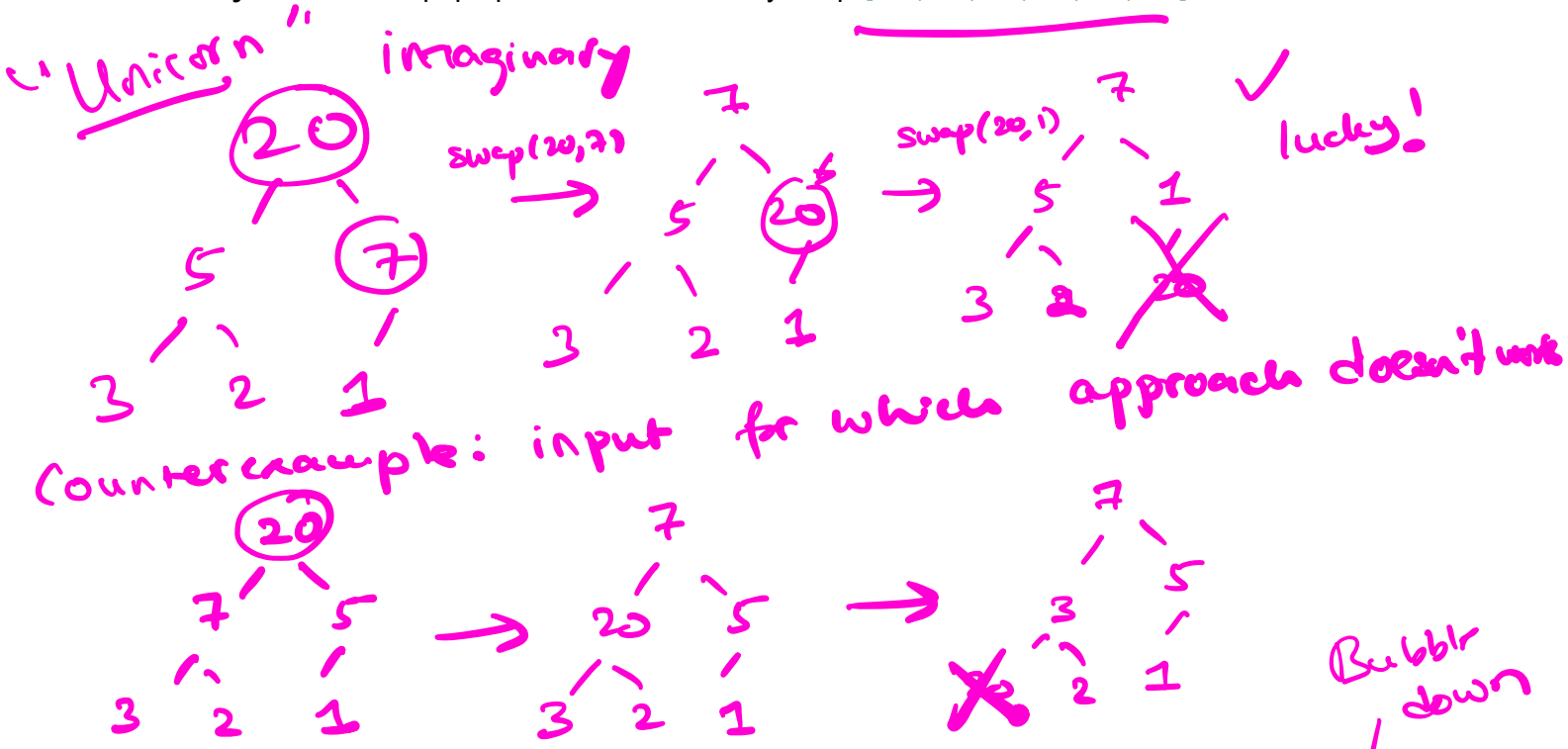
pop( ) — O(log n)

```
None
procedure pop():
    Step 1: Replace key of root with key of last node in the last level
    Step 2: Starting at root, "bubble down" until heap property is restored
```

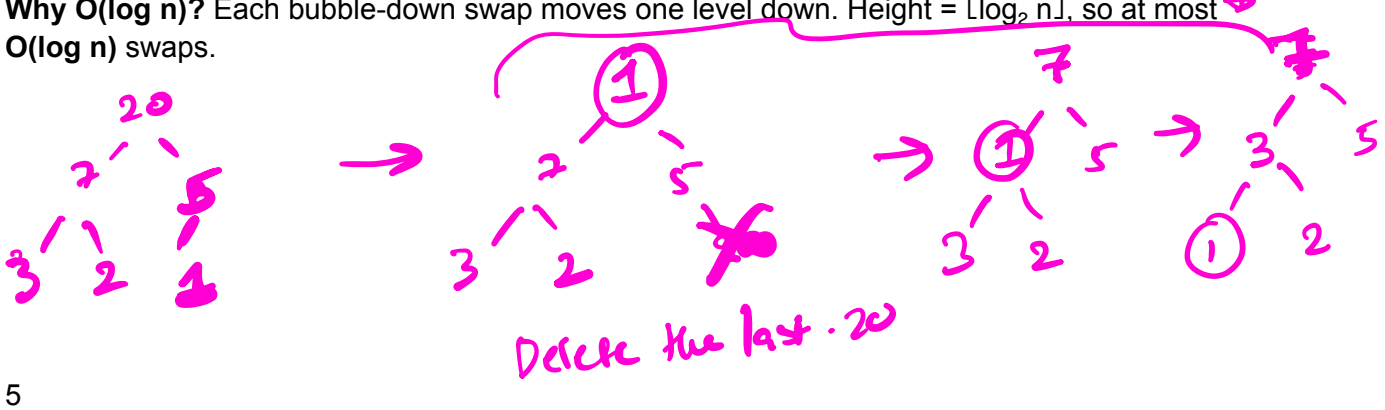
Bubble down (max-heap): swap with the **larger** child until heap property holds. **Bubble down (min-heap):** swap with the **smaller** child until heap property holds.

Why Step 1? If we just deleted the root, the tree would no longer be complete. By replacing it with the last node, we preserve the shape property in O(1) — then restore the heap property by bubbling down.

Activity 2 — Trace a pop operation on this binary heap `[20, 5, 7, 3, 2, 1]`.



Why O(log n)? Each bubble-down swap moves one level down. Height = $\lfloor \log_2 n \rfloor$, so at most **O(log n)** swaps.



4. Heapify: Build a Heap in O(n)

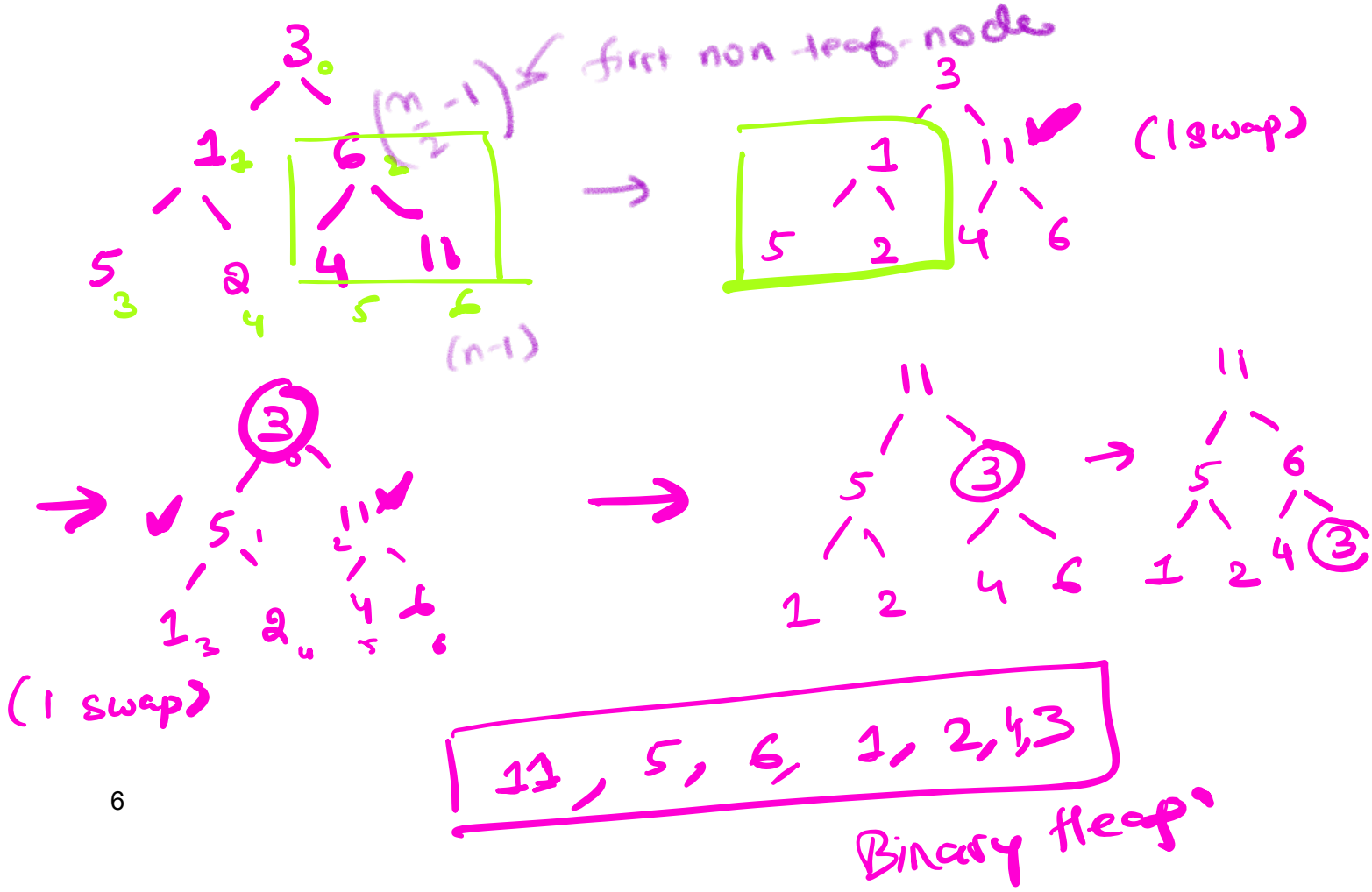
Problem: Given an arbitrary vector of n elements, convert it to a valid heap in-place.

Naive approach: call `push()` n times → O(n log n)

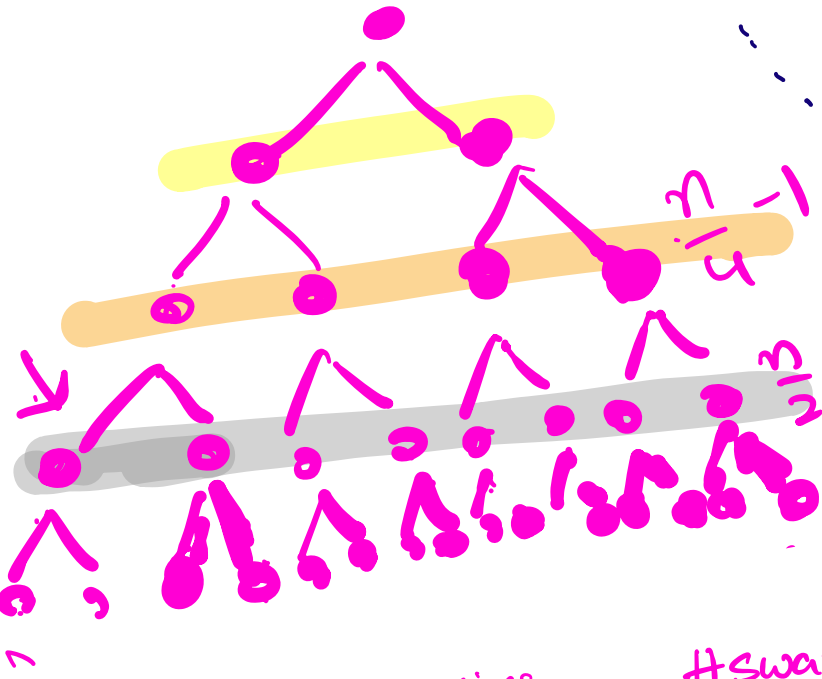
Better approach:

```
C/C++
void buildHeap(vector<int>& v) {
    int n = v.size();
    for (int i = n/2 - 1; i >= 0; i--) {
        bubbleDown(v, i);
    }
}
```

Consider the unsorted vector: *[3,1,6,5,2,4,11]*
Draw the binary tree representation, then trace buildHeap (or heapify) build a max heap.



What is the running time of



R

1 node needs h swaps

$\sim \frac{n}{8}$ nodes need atmost 2 swaps

$\sim \frac{n}{4}$ nodes need atmost 1 swap

Range of indices

$\frac{n}{2}$ to $(n-1)$

$\frac{n}{4}$ to $\frac{n}{2} - 1$

$\frac{n}{8}$ to $\frac{n}{4} - 1$

#swaps (i)

of nodes that need (i) swaps

0 → $n-1-\frac{n}{2}+1 = \frac{n}{2}$

1 → $\frac{n}{4}$

2 → $\frac{n}{8}$

k → $\frac{n}{2^{k+1}}$

h → $\frac{n}{2^{h+1}}$

$h = O(\log n)$

approximate to nearest whole number

Observe only root node needs at most h swaps to restore the heap property

$T(n) =$ c. * Total number of swaps in bubbkdown

$$= c. \left(1 * \frac{n}{4} + 2 * \frac{n}{8} + \cdots \cdot \frac{n}{2^{h+1}} \right)$$

$$= c. n \sum_{i=1}^{h} \frac{i}{2^{i+1}}$$

$\Downarrow$

$$= c.n. \text{ constant}$$

$$= O(n)$$

Summary

ADT	Priority Queue
Data structure	Binary Heap
Underlying representation	Complete Binary Tree stored as a Vector
`top()`	O(1) — root is always the best element
`push(x)`	O(log n) — append to vector + bubble up
`pop()`	O(log n) — swap with last + remove + bubble down
`buildHeap` (heapify)	O(n) — most nodes are leaves and do zero work

- The **heap property** is what makes `top()` O(1).
- The **complete binary tree shape** (stored as a vector) is what makes `push()` and `pop()` O(log n) — guaranteed, no degenerate case.

Next: Lab 03 — implement your own priority queue as a binary heap.